

# What is a transfoi

## Machine learning

Welcome

- ▶ Introduction
- ▶ Data science for machine learning
- ▶ Feature Engineering
- ▶ Supervised learning
- ▶ Unsupervised learning
- ▶ Semi-supervised learning
- ▶ Reinforcement learning
- ▼ Deep Learning
  - Overview
  - ▶ Neural networks
    - Recurrent neural networks
    - Long short-term memory (LSTM)
    - Convolutional neural networks
  - ▼ Transformer models
    - Overview**
    - Attention mechanism
    - Grouped query attention
    - Positional encoding



Hello! How can I help you?

mamba model

Graph neural network

- ▶ Generative AI
- ▶ Model training
- ▶ Machine learning libraries
- ▶ MLOps

By [Cole Stryker](#) , [Dave Bergmann](#)

*This article was featured in the Think newsletter*

# What is a transformer

The transformer model is a type of [neural network](#) that excels at processing sequential data, [large language models \(LLMs\)](#). Transformers have achieved elite performance in other fields of [artificial intelligence](#), such as computer vision, speech recognition

The transformer architecture was first described in the paper “[Attention Is All You Need](#)” [↗](#) by Vaswani and others, which is a key concept in [deep learning](#).

Originally introduced as an evolution of the [recurrent neural network \(RNN\)](#), sequence-to-sequence models used for machine translation have since attained cutting-edge advancements in the [machine learning \(ML\)](#) discipline.

Despite their versatility, transformer models are primarily used in the context of [natural language processing \(NLP\)](#), including [text summarization](#), question answering and sentiment analysis.

BERT (or Bidirectional Encoder Representation from Transformers) model introduced by Google in 2019, was a milestone in the world of transformers and remains the basis of most modern NLP applications.

Hello! How can I help you?

Autoregressive decoder-only LLMs, such as the (OpenAI GPT-3, Google PaLM, and OpenAI GPT-4) model that powered the launch of the new era of [generative AI \(gen AI\)](#).

The ability of transformer models to intricately process and correlate with the others also

For instance, vision transformers (ViTs) often complement [neural networks \(CNNs\)](#) on [image segmentation](#) tasks. The transformer architecture also powers many [diffusion models](#), [multimodal](#) text-to-speech (TTS) and [vision language models](#).

## Join over 100,000 subscribers to receive the latest news in tech

Stay up to date on the most important—and in-demand—tech news, data and beyond with the Think Newsletter, delivered weekly. [Read our Privacy Statement.](#)

First name\*

---

Last name\*

---

Business email\*

---

Your subscription will be delivered in English. You will receive one email per week. Refer to our [IBM Privacy Statement](#) for more information.



Hello! How can I help you?

# Why are transformer

The central feature of transformer models is the ability to capture dependencies between each part of an input sequence. Unlike recurrent neural networks (RNNs) and convolutional neural networks (CNNs), which process input sequentially, transformer models derive their impressive performance from self-attention mechanisms that allow them to process the entire input sequence at once. This is achieved through standard feedforward layers.

The benefits of self-attention, and specifically the way transformer models employ it to compute it, are what give them their superior performance over the RNNs and CNNs that had preceded them.

Before the introduction of transformer models, neural networks (RNNs). The way RNNs process sequences is by ingesting the elements of an input sequence one by one.

This hinders the ability of RNNs to capture long-range dependencies, meaning they can only process short text sequences effectively. This deficiency was somewhat addressed by recurrent neural networks (LSTMs) [\[1\]](#), but remains a fundamental limitation.

Attention mechanisms, conversely, can examine the entire input sequence and make decisions about how and when to focus on different parts of it.

In addition to significantly improving the ability to capture long-range dependencies, this quality of transformers also allows for parallel processing of computational steps at once, rather than in a sequential manner.

Being well-suited to parallelism enables transformers to leverage the raw power and speed offered by GPUs during both training and inference. This, in turn, unlocked the opportunity to train transformers on massive datasets through self-supervised learning.

Especially for visual data, transformers also offer a significant advantage over neural networks. CNNs are inherently local, using only a small portion of input data one piece at a time.

Therefore, CNNs also struggle to discern long-range dependencies between words (in text) or pixels (in images) that are far apart in the input sequence.

Hello! How can I help you?

# What is self-attention

Understanding the mathematical concept of a self-attention mechanism is essential to understanding the success of transformer models. Attention mechanisms are, in essence, algorithms that allow a model to focus on specific parts of a data sequence. An AI model should “pay attention” to the most relevant information in a data sequence.

Consider a language model interpreting the sentence: “On Friday, the judge issued a sentence.”

- The preceding word “the” suggests that the judge is *a person presiding over a legal trial*—rather than a *legal penalty*.
- That context for the word “judge” suggests a *legal penalty*, rather than a grammatical “subject”.
- The word “issued” further implies that the judge is *issuing a legal penalty*.
- Therefore, when interpreting the word “sentence,” the model should pay *attention* to “judge” and “issued.” It can more or less ignore the other words in the sentence.

## How does self-attention

Broadly speaking, a transformer model’s self-attention mechanism processes the context of each part of a data sequence in 4 steps:

1. The model “reads” raw data sequences and converts them into vectors, in which each element in the sequence is represented by a vector that numerically reflects qualities such as semantic meaning.
2. The model determines similarities, correlations (or *inner products* thereof) between each vector and each other vector in the sequence. The relative importance of one vector to another is determined by the *product* between each vector. If the vector product yields a large value, the vectors are aligned. If they’re not aligned, the vectors are not aligned.
3. These “alignment scores” are converted into *attention weights* as inputs to a *softmax* activation function.

Hello! How can I help you?

Vector B should be ignored when making predictions.

an attention weight of 1 means that it should focus on Vector A when making decisions about Vector A.

4. These attention weights are used to emphasize specific input elements at specific times. In other words, transformer models focus on or ignore specific information based on the current context.

Before training, a transformer model doesn't have any learned embeddings and alignment scores. During training, it processes millions of examples drawn from its training dataset to learn each prediction.

Through an iterative cycle of making predictions and adjusting alignment scores and attention weights that lead to the best results, [backpropagation](#) and [gradient descent](#), the model learns to predict the next token in a sequence.

## How do transformer models work?

[Transformer models](#) such as [relational databases](#) process each part of a data sequence, and use them to calculate the results of matrix multiplications.

Relational databases are designed to simplify data processing; they assign a unique identifier ("key") to each part of a data sequence, and use them to calculate the results of matrix multiplications. The "Attention is All You Need" framework to processing the relationships between different parts of a data sequence.

- The *query vector* represents the information about the current words, a token's query vector is used to compare the current word's meaning, conjugation or connotations in context with the other words in the sequence.
- The *key vectors* represent the information about the other words in the sequence; the query and key is used to compute attention weights for the context of that text sequence.
- The *value (or value vector)* "returns" the information about the other words in the sequence; the respective attention weight. Contributions from the other words are weighted more heavily; contributions from words that are further away from the query will be weighted closer to zero.

For an LLM, the model's "database" is the user's input.

Hello! How can I help you?

## Tokenization and input embeddings

Whereas *characters*—letters, numbers or punctuation—use to represent language, the smallest unit of text (a token is assigned an ID number, and these ID numbers themselves) are the way LLMs navigate language. *tokenization* of language significantly reduces the size of text.

To generate query and key vectors to feed into the model, the model needs an initial, contextless vector embedding. These embeddings can be either learned during training or generated by an embedding model.

## Positional encoding

The order and position of words can significantly impact the meaning of the sentence. Unlike the serialized nature of RNNs, which inherently process one token at a time, transformer models must explicitly address the order of tokens. A positional encoding mechanism is used to consider the position of each token in the sequence.

With *positional encoding*, the model adds a vector to the input embedding derived from its relative position, before the input is processed by the model. The closer the two tokens are, the more similar their positional encodings will be, and their alignment score will increase from there. The model thereby learns to pay greater attention to nearby tokens.

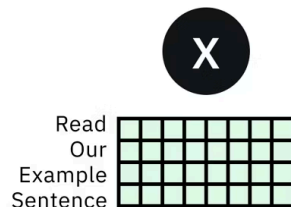
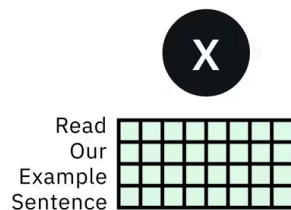
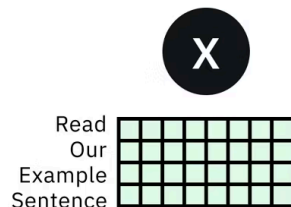
## Generating query, key and value vectors

When positional information has been added, the model generates three new vectors: *query*, *key*, and *value*. These are generated by passing the original token embeddings through each of the layers that precede the first attention layer. Each layer has a unique matrix of weights, learned through self-supervised learning on text.

- The embeddings are multiplied by the weights to generate query, key, and value vectors, which have  $d_k$  dimensions

Hello! How can I help you?

- The embeddings are multiplied by the weights with dimensions  $d_v$



A diagram of a transformer model's attention mechanism

## Computing self-attention

The transformer's attention mechanism's primary goal is to assign weights to the pairings of each token's query vector with all other tokens in the sequence. When achieved, you can calculate the corresponding vector of attention weights, which is then multiplied by the value vector of each token to produce the final output.

Hello! How can I help you?

- Each other token's *value vector* is now multiplied by the attention weights to produce the final output.



- These attention-weighted value vectors are aggregated to represent the aggregated contextual information from all other tokens in the sequence.
- Finally, the resulting vector of attention-weighted values is added to the token  $x$ 's original, post-positional encoding to produce the final output.

In essence,  $x$ 's vector embedding has been updated with information from all other tokens in the sequence.

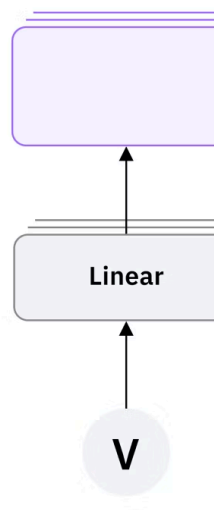
## Multi-head attention

To capture the many multifaceted ways tokens relate to each other, transformer models implement [multi-head attention](#) across the input and output sequences.

Before being fed into the first feedforward layer, the input and output sequences are split into  $h$  evenly sized subsets. Each piece of the sequence is then multiplied by matrices of  $Q$ ,  $K$  and  $V$  weights, each of which represents a different *head*. The vectors output by each of these parallel operations are then fed into a corresponding subset of the next layer's feedforward network.



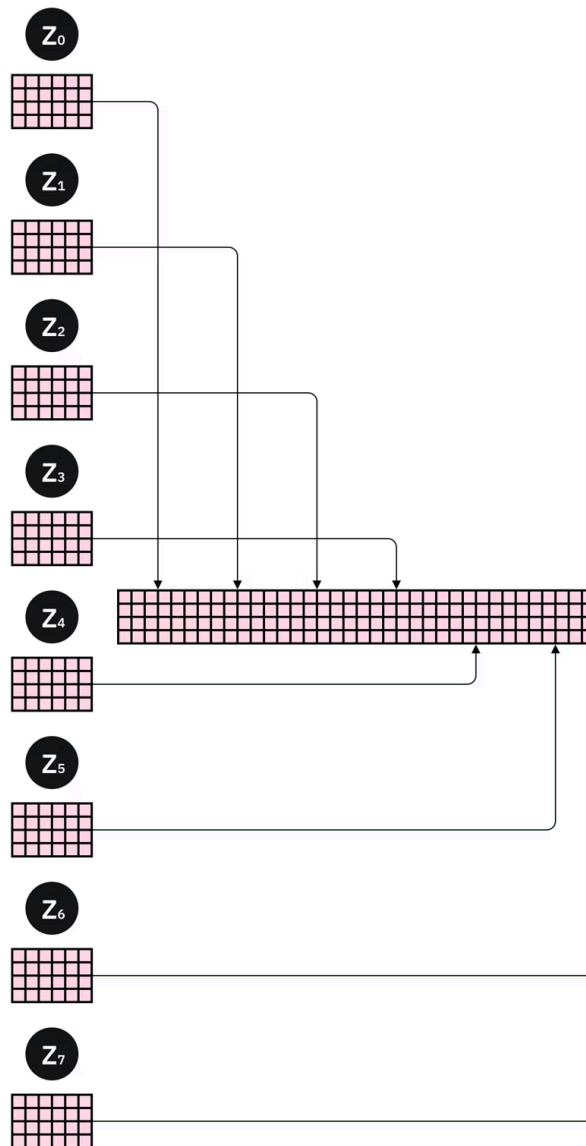
Hello! How can I help you?



The simplified multi-head attention diagram made

In the final layers of each attention block, the eventually concatenated back together before practice, model training results in each circuit separate aspect of semantic meanings.

Hello! How can I help you?



The outputs "Z" of each attention head are concatenated

## Residual connections and layer norm

In some situations, passing along the context through an attention block might result in an unacceptable sequence.

To address this, transformer models often balance the attention mechanism with the original sequence. This is done by adding residual connections that bypass the attention block, allowing the original sequence to be combined with the attention-updated subsets of the token embeddings.

Hello! How can I help you?

The resulting vector is fed into another linear layer to a constant size before being passed along the next layer. These measures help preserve stability in training and ensure information is not lost as the data moves deeper into the network.

## Generating outputs

Eventually, the model has enough contextual information that the nature and function of the output layer will depend on the model has been designed for.

In autoregressive LLMs, the final layer uses a softmax layer that the next word will match each token in its output. Based on specific [sampling hyperparameters](#), the model generates the next token of the output sequence.

# Transformer models processing (NLP)

Transformer models are most commonly associated with being developed for machine translation use cases. However, they gave rise to the large language models (LLMs)

Most of the LLMs that the public is most familiar with, such as OpenAI's [GPT](#) series and Anthropic's [Claude](#) or Llama or [IBM® Granite®](#), are autoregressive decoders.

Autoregressive LLMs are designed for text generation and adjacent tasks such as summarization and question answering. They use self-supervised learning, in which the model is trained by being tasked with iteratively predicting the next word in a sequence.

Information provided by the self-attention mechanism allows the model to capture context from the input sequence and maintain coherence across long distances.

Encoder-decoder masked language models (Masks) represent the other main evolutionary branch of LLMs.

Hello! How can I help you?



While this training methodology is less effective for tasks requiring robust contextual information, learning embeddings.

# Transformer models

Though transformer models were originally developed and prominently associated with natural language processing, the situation involving sequential data. This has led to transformer models in other fields, from [fine-tuning LLMs](#) to [series forecasting models](#) and ViTs for computer vision.

Some data modalities are more naturally suited to sequential representation than others. Time series, audio, and video are, whereas image data is not. Despite this, ViTs have achieved state-of-the-art results for many computer vision tasks, including [captioning](#), [object detection](#), [image segmentation](#), and [image classification](#).

To use transformer models for data not conventionally thought of as "sequential" requires a conceptual workaround to represent that data as a sequence. For instance, to use attention mechanisms to understand visual data, ViTs use *patch embeddings* to make image data interpretable as sequences.

- First, an image is split into an array of patches. For instance, a 224x224 pixel image can be subdivided into 256 14x14 pixel patches, dramatically reducing the number of computational steps required to process the image.
- Next, a linear projection layer maps each patch to a [vector embedding](#).
- Positional information is added to each of these patch embeddings, akin to the positional encoding described earlier in this article.
- These patch embeddings can now essentially function as a sequence of token embeddings, allowing the image to be interpreted by an attention mechanism.

## Authors

Hello! How can I help you?



**Cole Stryker**

Staff Editor, AI Models  
IBM Think



**Dave Bergmann**

Senior Staff Writer, AI Models  
IBM Think



## Resources

Explainer

### Neural networks from scratch

Get an in-depth understanding of neural networks, their basic functions and the fundamentals of

Hello! How can I help you?

AI models

## Explore IBM Granite

IBM Granite® is a family of open, high-performance, and trusted AI models designed for business and optimized to scale your AI applications. Explore options across language, code, time series and guardrails.

[Meet Granite](#)



Ebook

## Unlock the power of generative AI and ML

Learn how to confidently incorporate generative AI and machine learning into your business.

[Read the ebook](#)



Hello! How can I help you?

Ebook

Guide

## The CEO's guide to generative AI

Learn how CEOs can balance the value generative AI can create against the investment it demands and the risks it introduces.

[Read the guide](#)



Guide

## Put AI to work: Driving ROI with gen AI

Want to get a better return on your AI investments? Learn how scaling gen AI in key areas drives change by helping your best minds build and deliver innovative new solutions.

[Read the guide](#)



1 / 2



## Related solutions

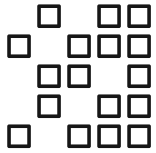
Hello! How can I help you?



**IBM® watsonx Orchestrate™**

Easily design scalable AI assistants and agents, automate repetitive tasks and simplify complex processes with IBM® watsonx Orchestrate™.

[Explore watsonx Orchestrate](#) →

**AI for developers**

Move your applications from prototype to production with the help of our AI development solutions.

[Explore AI development tools](#) →

**AI consulting and services**

Reinvent critical workflows and operations by adding AI to maximize experiences, real-time decision-making and business value.

[Explore AI services](#) →

## Take the next step

Whether you choose to customize pre-built apps and skills or build and deploy custom agentic services using an AI studio, the IBM watsonx platform has you covered.

[Explore watsonx Orchestrate](#)

[Explore watsonx.ai](#)



Hello! How can I help you?

## Footnotes

<sup>1</sup> [Google's BERT Rolls Out Worldwide](#) . Search Engine Journal. December 9, 2019.

## Footnotes

<sup>1</sup> [Google's BERT Rolls Out Worldwide](#) (external link to ibm.com). Search Engine Journal. December 9, 2019.

Hello! How can I help you?